CLOUDGUARD

Teammate Role & Action Plan

Project handover • Based on the current implementation audit • 27 October 2026

Purpose

I started the CloudGuard implementation and the current repository already contains both a Python backend and a web dashboard. I will take ownership of the backend/security logic and the existing backend behavior. Your role is to take ownership of the frontend review, UI integration/quality, and the supporting case-study presentation work. This division is intended to avoid duplicate implementation and keep the team aligned to the final PRD.

Project context

CloudGuard is a local-only CNS educational prototype based on selected technical concepts from the 2019 Capital One cloud data breach. It uses mock services and synthetic customer records to demonstrate a vulnerable configuration versus a secured configuration. It does not connect to AWS or reproduce the original production environment.

Ownership split

Owner	Responsibilities	Expected deliverables
Me — Backend & security logic	Own server.py; scenario state; mock SSRF/metadata/IAM/S3 logic; policy evaluation; remediation behavior; backend tests and API correctness.	Reliable backend endpoints; verified access decisions; security logic tests; backend notes and documented limitations.
You — Frontend & presentation	Review and own static/index.html, CSS and frontend JS; verify buttons/tabs/visual states; ensure UI reflects backend responses; prepare screenshots, case-study explanation and demo flow.	Polished and accurate UI; browser test checklist; screenshots; presentation/demo script; incident-to-control explanation.
Shared	Review the PRD together; agree API contracts; run end-to-end tests; review code and documentation; both prepare for viva.	Integrated, tested prototype and consistent final submission.

Current repository context

The audit reports the following existing components. Please inspect and improve them rather than rebuilding them from scratch unless a specific defect justifies a change.

- **Backend:** server.py — Python standard-library HTTP server and simulated security services.
- **Frontend:** static/index.html, static/css/style.css, static/js/app.js.
- **Interactive modules:** static/js/topology.js and static/js/terminal.js.
- **Data:** data/policies.json, data/dummy_customers.json and data/breach_info.json.
- **Tests:** tests/test_simulation.py — audit reports 12 passing tests; browser interactions are not covered by that suite.

Known issues and assigned follow-up

Issue	Owner	Action
Reset button resets UI but not backend scenario	Me + you verify interface	I will correct backend/state behavior; you verify the UI reflects the reset response.
Accidental global assignment in terminal.js	You — frontend	Correct the request body assignment and verify terminal commands still work.
IMDSv2 token handshake test missing	Me — backend	Add test for token issuance and token-authenticated request, using local mock only.
S3 bucket-policy isolation test missing	Me — backend	Add a test showing the bucket policy independently denies access when IAM is otherwise broad.
Browser interactions not covered by Python tests	You — frontend	Create and execute a manual browser checklist for tabs, scenario switch, run/reset and API-driven status.

Issue	Owner	Action
Audit logs are in memory	Joint decision	For the basic scope, document the limitation; only add persistence if time and rubric justify it.

Non-negotiable scope: follow the final PRD

Please keep development aligned with CloudGuard_Final_PRD_and_Implementation_Status.pdf. Do not add new modules or change the architecture without discussing it first. The project is intentionally a basic local simulation, not a full cloud-security platform.

- No real AWS deployment, accounts, credentials or cloud API calls.
- No external-target SSRF testing or real metadata endpoint requests.
- Synthetic customer records only.
- The terminal is a simulator, not a real shell; topology animations are illustrative.
- Preserve working features; make small changes with a clear reason and test evidence.

Milestone workflow

- M00 — Workspace audit: completed; the audit has been supplied.
- M01 — PRD and interface contract: finalize/commit the PRD and agree on ownership/API expectations.
- M02 — Existing UI review: verify current screens and API integration; do not rebuild the dashboard.
- M03 — Backend verification and fixes: implement assigned backend test/fix items.
- M04 — End-to-end validation: verify vulnerable versus secured results and reset behavior.
- M05 — Documentation and demo: finalize README, screenshots, report and short presentation.

Definition of done

- The same mock unauthorized request succeeds only in the vulnerable model and is denied in the secured model.
- UI labels and status are driven by backend responses where applicable.
- Reset leaves frontend and backend in the same documented state.
- Automated and manual tests are recorded honestly, including failures and limitations.
- Both members can explain SSRF, least privilege, metadata protections, storage policy and why the model is simplified.

Important: This handover assigns me ownership of the backend/security logic because I initiated the implementation and the existing backend is already present. You are not expected to rebuild the entire frontend; your responsibility is to verify, refine and document the UI and presentation side in line with the PRD.